



An empirical study of developers' challenges in implementing Workflows as Code: A case study on Apache Airflow[☆]

Jerin Yasmin, Jiale Amber Wang, Yuan Tian^{*}, Bram Adams

School of Computing, Queen's University, Kingston, ON, Canada

ARTICLE INFO

Dataset link: https://github.com/RISElabQueen/s/dataset_developers_challenges_WaC

Keywords:

Workflows as Code
Apache Airflow
Workflow orchestration
Stack overflow
Empirical study

ABSTRACT

The Workflows as Code paradigm is becoming increasingly essential to streamline the design and management of complex processes within data-intensive software systems. These systems require robust capabilities to process, analyze, and extract insights from large datasets. Workflow orchestration platforms such as Apache Airflow are pivotal in meeting these needs, as they effectively support the implementation of the Workflows as Code paradigm. Nevertheless, despite its considerable advantages, developers still face challenges due to the specialized demands of workflow orchestration and the complexities of distributed execution environments. In this paper, we manually study 1000 sampled Stack Overflow posts derived from 9591 Airflow-related questions to understand developers' challenges and root causes while implementing Workflows as Code. Our analysis results in a hierarchical taxonomy of Airflow-related challenges that contains 7 high-level categories and 14 sub-categories. We find that the most significant obstacles for developers arise when defining and executing their workflow. Our in-depth analysis identifies 10 root causes behind the challenges, including incorrect workflow configuration, complex environmental setup, and a lack of basic knowledge about Airflow and the external systems that it interacts with. Additionally, our analysis of references shared within the collected posts reveals that beyond the frequently cited Airflow documentation, documentation from external systems and third-party providers is also commonly referenced to address Airflow-related challenges.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

1. Introduction

The advent of large-scale data has revolutionized the design and development of software systems. New software systems, characterized by their intensive data processing capabilities, are increasingly pivotal across various sectors, including healthcare, finance, manufacturing, and the automotive industry, among others (Keresting et al., 2020). We refer to these systems as *data-intensive software systems*.

Data-intensive software systems often involve executing a series of interconnected activities, collectively forming a *workflow*. These workflows are complex, encompassing various components like code, data from multiple sources, specific configurations (such as execution conditions), and the necessary infrastructures (Sculley et al., 2015; Munappy et al., 2019). The specific activities within these workflows vary depending on the application context. For instance, an ETL (Extract, Transform, Load) workflow typically involves activities that begin with extracting data from various sources. This data is then transformed

into a usable format before being loaded to its final destination for direct user access or subsequent processing tasks. In the context of Machine Learning (ML)-enabled software systems, workflows typically include activities such as loading data from various sources, preprocessing the data to prepare it for analysis, engineering features, executing machine learning algorithms, validating, deploying the solution, and continuously monitoring its performance (Amershi et al., 2019; C. W. Danilo Sato, 2019).

The manual development and management of these workflows tends to be both time-consuming and prone to errors. This is largely due to their reliance on complex execution conditions, the handling of large-scale data, and the necessity for iterative development and experimentation (Munappy et al., 2020; Sculley et al., 2014). For instance, while some workflows are designed for daily execution, others are activated by specific triggers, such as the arrival of new data. This diversity requires not just accurate scheduling, but also adaptable systems that can smoothly incorporate new data. The challenges are

[☆] Editor: Gabriele Bavota.

^{*} Corresponding author.

E-mail address: y.tian@queensu.ca (Y. Tian).

manifold, encompassing the potential for execution errors, difficulties in maintaining the accuracy of large datasets, and the need for continuous adaptation to optimize performance.

In response to these challenges, the paradigm of “*Workflows as Code*” has emerged as a programmable approach, enabling orchestration (defining, scheduling, and monitoring) of workflows using code. This approach integrates software engineering principles into workflow development and management, offering several advantages over traditional methods. By treating workflows as code, organizations can leverage automation and code-driven management, enhancing the development, update, and integration processes of workflows with other tools and platforms. As such, there has been a notable shift towards embracing the Workflows as Code paradigm in recent years. This has led to a new generation of orchestration platforms such as Apache Airflow,¹ Luigi,² and Dagster.³ Developers have increasingly adopted these platforms to implement Workflows as Code, empowering them to streamline operations and enhance collaboration across teams.

While the Workflows as Code paradigm has enabled efficient workflow development, it also introduces new challenges. The flexibility in coding methods can lead to workflows that are difficult to launch or debug, compounded by the lack of established best practices for development. Moreover, different from traditional systems, developers might require a steep learning curve to understand the concept of workflow development and management (Yackel, 2021). Additionally, in complex data processing scenarios involving numerous datasets, transformations, and conditional logic, it can be hard to see how data flows and what causes problems or delays. Despite these potential challenges, no prior research has investigated the implementation of Workflows as Code in software development.

To fill this knowledge gap, this paper presents an empirical study to understand the types and root causes of challenges developers face while implementing Workflows as Code leveraging workflow orchestration platforms. The result of such a study will help developers quickly understand the fundamental difficulties and common pitfalls associated with Workflows as Code. For our study, we focus on Airflow, which has recently risen to prominence as a premier open-source orchestration platform, significantly simplifying both the development and management of workflows in data-intensive software systems (Star-History, 2024). World-leading corporations, including Adobe, Astronomer, Etsy, Google, ING, Paypal, Qubole, Quizlet, Reddit, Square, and Twitter, have embraced Airflow for managing intricate software tasks, especially those workflows in their data-intensive systems (Sally, 2019; John Thomas, 2022).

In line with common practices of using question-and-answer sites, such as Stack Overflow (SO), to explore the challenges developers face in specific software engineering tasks (Yang et al., 2016; Rosen and Shihab, 2016; Ahmed and Bagherzadeh, 2018; Alshangiti et al., 2019; Bagherzadeh and Khatchadourian, 2019; Wang et al., 2022), we gathered a dataset of 9591 Airflow-related questions and their corresponding answers from SO. Utilizing this data, we conduct an empirical investigation to address three research questions:

- **RQ1: What are the types of challenges developers face in using Airflow?** We sampled a subset of 1000 Airflow-related questions and identified a comprehensive hierarchical taxonomy of challenges faced by developers, encompassing 7 high-level categories and 14 subcategories. The most common inquiries revolve around seeking assistance in defining workflows and tackling problems encountered during workflow execution. Developers also face other difficulties, such as setting up deployment and production environments for Airflow and ensuring the quality of workflows.

- **RQ2: What are the root causes of the challenges?** We examined each post considered in RQ1 along with corresponding accepted or highest-voted answers and comments, and identified the root causes of the identified challenges. We identified 10 types of root causes underlying the challenges that developers encounter with Airflow. These root causes highlight a two-fold issue: on the one hand, Airflow’s official documentation does not sufficiently guide users; on the other hand, there is a lack of developers’ understanding and knowledge regarding Airflow, its interactions with external systems, as well as broader aspects of programming and DevOps practices.
- **RQ3: What are the types of online sources developers refer to in Airflow-related posts?** We extracted 16,059 references of URLs shared by developers within the 9591 collected posts. We observed that the most frequently referenced resource is Airflow’s official documentation. However, our analysis also shows a significant reliance on documentation from external systems and third-party resources. This trend indicates the diverse range of information sources developers consult to address Airflow-related challenges.

Our research uncovers the types of common problems in implementing Workflows as Code. More specifically, this paper makes the following main contributions:

- We are the first to explore the challenges faced by developers when implementing Workflows as Code using Apache Airflow. We propose two taxonomies to characterize these challenges and their root causes. We also discuss their prevalence.
- We explore how online documentation resources with different relevance to Airflow are referred to via shared links and present our observations.
- We discuss the implications of our findings from both research and practice perspectives and provide a replication dataset⁴ with annotated SO posts at different levels from various aspects (challenge, root cause, references) to foster future research.

The rest of the paper is organized as follows: Section 2 discusses the background of Airflow and related work. Section 3 presents the data collection overview and Section 4 performs the preliminary studies. Section 5 details the three research questions. Section 6 presents discussions on our findings. Section 7 discusses the threats to validity and finally, Section 8 concludes the paper.

2. Background and related work

2.1. Background

2.1.1. Platforms supporting Workflows as Code

With the rise of data-intensive products and decision-making, open-source software engineering platforms embracing the Workflows as Code paradigm have become extremely popular in the last five years.

Airflow, Prefect,⁵ Dagster,⁶ Luigi,⁷ Metaflow⁸ and Flyte⁹ enable workflow definitions in Python. Prefect and Dagster, supported by their cloud offerings, Prefect Cloud and Dagster Cloud allow practitioners to define end-to-end data pipelines and ML pipelines. Metaflow, developed by Netflix, can be used to define DAG-based data and ML pipelines. It provides versioning of artifacts, reproducibility, and native support for experimentation (Berg et al., 2019). Motivated by the

¹ <https://airflow.apache.org/>

² <https://github.com/spotify/luigi>

³ <https://dagster.io/>

⁴ https://github.com/RISElabQueens/dataset_developers_challenges_WaC

⁵ <https://www.prefect.io/>

⁶ <https://dagster.io/>

⁷ <https://github.com/spotify/luigi>

⁸ <https://metaflow.org>

⁹ <https://flyte.org/>

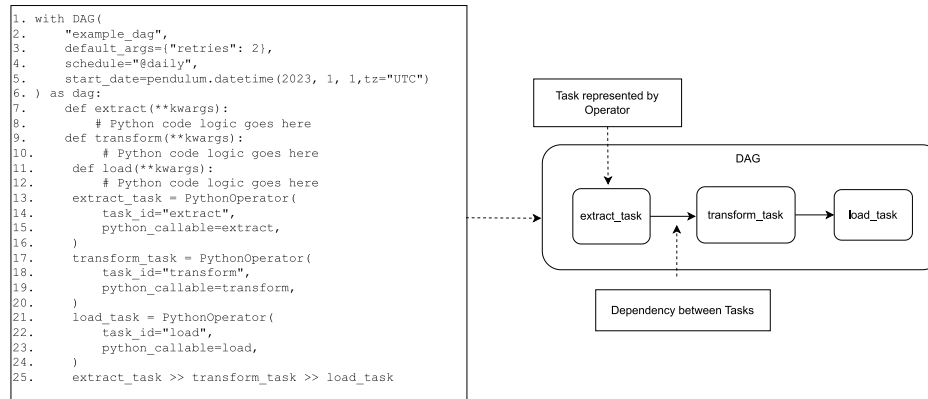


Fig. 1. “Workflows as Code” through Directed Acyclic Graphs (DAGs), visually capturing the seamless orchestration of tasks and dependencies in a concise, efficient manner.

“model card” concept (Mitchell et al., 2019), researchers have developed a prototype tool on top of Metaflow that generates documentation of ML pipelines termed as “DAG card” (Tagliabue et al., 2021). Flyte, developed by Lyft, is a pipeline automation platform for complex data and ML pipelines at a scale where pipelines can be defined using their REST/gRPC API with SDK support for Java, Python and Scala.

Argo,¹⁰ Azkaban,¹¹ and Pachyderm¹² enable workflows to be defined via JSON, YAML, or XML files. Argo natively runs parallel jobs on Kubernetes, an open-source container orchestration system that automates software deployment, scaling, and management. Azkaban is a batch workflow job scheduler created at LinkedIn to run Hadoop jobs. Data-driven pipelines are defined using Pachyderm, where data changes initiate the trigger. It provides data lineage with data versioning, and its usage ranges from complex data processing systems and ML system development to research in biotech and life sciences (Novella et al., 2019). These platforms differ in their supported code formats, core functionalities, integration capabilities, and target audience.

In our research, we have chosen Apache Airflow from among the platforms that support Workflows as Code. This decision was made based on Airflow’s widespread adoption within the industry and its popularity on Stack Overflow. A detailed discussion is elaborated in Section 7.

2.1.2. Airflow

Airflow, initially developed by Airbnb in 2014, has become a widely adopted open-source workflow orchestration tool for the development, scheduling, and monitoring of workflows. It entered the Apache incubator in 2016 and achieved the status of a top-level project under the Apache Software Foundation in 2019 (Sally, 2019). Renowned for its proficiency in managing data-intensive workflows, Airflow has found extensive application in areas such as ETL processes, where the complexity of workflows necessitates robust management and automation. In the rest of this subsection, we describe how a workflow is defined using Airflow and the architecture and ecosystem of the platform.

Workflows as Code using Airflow. Airflow positions itself as a platform embodying the Workflows as Code paradigm.¹³ In Airflow, workflows are constructed using Python scripts, which outline *Directed Acyclic Graphs (DAGs)*. These graphs are “directed” - indicating a predetermined order of task execution, and “acyclic” - guaranteeing the absence of cycles or loops within the workflow. In each DAG script, the tasks are represented as nodes, with the dependencies between these tasks depicted as edges. The most commonly used approach for implementing tasks is through *operators*, as they offer a versatile and

straightforward method for defining various types of tasks. Operators encapsulate the logic necessary to perform specific actions, such as running a Python function, executing a Bash command, transferring data between systems, or interacting with external services like databases or cloud platforms. Besides operators, Airflow also offers alternative approaches such as the TaskFlow API (Airflow Documentation, 2024b) or custom operator implementation for more specialized requirements.

Fig. 1(left) illustrates an example of a workflow defined in Airflow using a DAG named “example_dag” (Line 2). This DAG has three parameters (Line 3–5). The “default_args” parameter is used to set common parameters for the DAG, such as defining the number of retries as 2. The “schedule” parameter and the “start_date” parameter determine that this DAG is scheduled to execute daily, commencing from January 1, 2023, in Coordinated Universal Time (UTC).

The sample workflow shown in Fig. 1 (left) consists of three tasks: “extract_task”, “transform_task” and “load_task” (Line 13–24). These tasks are responsible for the essential operations in ETL workflow: data extraction, transformation, and loading, respectively. Each task is based on a Python function, i.e., extract, transform, and load (Line 7–12). Each task is executed via a PythonOperator (Airflow Documentation, 2024a), a built-in operator provided by Airflow to execute Python callables. The Python functions act as placeholders for actual Python code, which will be executed during the respective task runs in the DAG. For simplicity of the example, we omit the detailed source code that developers should write to implement the corresponding logic. The three tasks are structured in a linear sequence (Line 25), ensuring that the output of one task is the input for the next.

Fig. 1 (right) shows an abstract of the sample workflow. This graphical representation (a simple linear relationship in this case) showcases the sequence of tasks and their dependencies. In practice, the dependencies between tasks in a workflow can indeed become more complex than what is depicted in the abstract representation shown in Fig. 1 (right). For instance, tasks may have conditional dependencies, where their execution depends on the outcome of other tasks or external factors. Additionally, tasks may have parallel dependencies, where multiple tasks need to be completed before proceeding with subsequent tasks.

Airflow Components. Airflow’s architecture is composed of a *scheduler*, *web server*, *metadata database*, *executor*, and *workers*. We briefly discuss each element below:

- **Metadata Database:** Airflow uses a metadata database to store information about DAGs, tasks, and their status. This allows for tracking and managing the state of workflows.
- **Web Server:** Airflow provides a web-based user interface to monitor and manage workflows. Users can visualize DAGs, view task logs, and manually trigger or pause DAG runs.

¹⁰ <https://argoproj.github.io/workflows/>

¹¹ <https://azkaban.github.io/>

¹² <https://www.pachyderm.com/>

¹³ <https://airflow.apache.org/docs/apache-airflow/stable/index.html>

- **Scheduler:** The scheduler monitors all tasks and DAGs to decide what needs to be run, based on dependencies and schedules. It sends tasks to be executed to the executor. Different types of executors are available in Airflow including the LocalExecutor, which handles tasks locally for parallel processing, and the SequentialExecutor, ideal for sequential task execution in simpler setups. For distributed task execution, Airflow provides the CeleryExecutor, utilizing Celery for task distribution across multiple machines, and the KubernetesExecutor, which scales tasks by creating separate pods within a Kubernetes cluster. In environments designed for distributed and parallel execution, tasks are allocated across various workers, i.e., the computational units responsible for executing tasks, enabling them to be executed simultaneously.
- **A folder of DAG files:** DAGs are typically stored as Python files in a designated folder. The scheduler monitors this folder, reading the DAG files to understand the task dependencies and execution schedules, and determine what tasks to run and when to run them, thus ensuring the smooth execution of workflows.

The Airflow Ecosystem. The Airflow ecosystem has evolved to support fully managed workflow orchestration services such as Google Cloud Composer and Amazon Managed Workflows for Apache Airflow (MWAA). Google Cloud Composer offers a managed Apache Airflow service on Google Cloud Platform (GCP), facilitating seamless workflow orchestration with native GCP integrations and automatic updates. Similarly, MWAA provides a managed Apache Airflow environment on AWS, offering features like auto-scaling and integration with AWS services. Astro, provided by Astronomer, offers a modern data orchestration platform powered by Apache Airflow, enabling data engineers, scientists, and analysts to build, run, and observe Workflows as Code. Azure Data Factory Managed Airflow and Yandex Managed Service for Apache Airflow offer similar managed services on Azure and Yandex Cloud platforms, respectively. Additionally, solutions like Airflow with Restack and DoubleCloud Managed Service for Apache Airflow provide flexibility by enabling users to deploy Apache Airflow on their preferred cloud infrastructure or managed platforms.

2.2. Related work

2.2.1. Challenges on workflow development and management

To the best of our knowledge, there has been no prior research investigating the challenges and implementation practices of Workflows as Code. The closest related studies focus on the difficulties encountered in the manual development and management of workflows within data-intensive systems.

Munappy et al. (2020) interviewed developers from the telecommunication and automobile domains and revealed three primary challenges in data pipeline management: infrastructure, data quality, and organizational issues. The authors highlighted the critical need for standardization across data pipelines, enhanced traceability, and the adoption of DataOps practices to address these challenges. Another work from the same authors (Munappy et al., 2019) conducted semi-structured interviews with seven deep learning (DL) experts. They identified the data management challenges in DL systems across seven stages, i.e., data collection, exploration, preprocessing, dataset preparation, data testing, deployment, and post-deployment. Polyzotis et al. (2018) identified fundamental challenges in ML data management that are applicable across various ML platforms, based on insights gained from constructing data management infrastructure for TFX. They highlight three main challenges: understanding the data, validating and cleaning it, and preparing data for ML tasks.

The existing reported challenges in manual workflow development and management underscore the need for adopting Workflows as Code. This programmable approach could facilitate traceability and standardization across data and machine learning pipelines.

2.2.2. Studies on developers' faced challenges leveraging Stack Overflow

Stack Overflow (SO) serves as a valuable platform for understanding software engineering practices from developers' perspectives. Researchers have conducted empirical studies leveraging SO, and these studies typically fall into two categories: studies that analyze the general types of questions asked by developers on SO and those that delve into domain-specific challenges reflected in the questions.

Previous research (Allamanis and Sutton, 2013; Beyer et al., 2020; Cummaudo et al., 2020) has identified general types of inquiry on SO, including conceptual questions, how-to guides, API usage, error troubleshooting, and discrepancies, shedding light on the prevalent topics of discussion. However, although these taxonomies provide a useful framework for understanding general programming queries, they do not capture the nuanced challenges developers face within specialized domains.

Domain-specific research has been conducted across various fields, leading to the creation of taxonomies describing challenges and underlying root causes in specific application domains. For instance, an analysis of 2,758 questions related to Puppet identified 16 challenge categories specific to configuration as code (Rahman et al., 2018). Similarly, focused studies have been conducted in numerous domains, including Apache Spark (Wang et al., 2022), machine learning (Alshangiti et al., 2019), big data (Bagherzadeh and Khatchadourian, 2019), mobile development, deep learning development (Zhang et al., 2019) and deep learning deployment (Chen et al., 2020), each revealing unique challenges pertinent to their respective areas.

In Section 6, we compare our taxonomies to the empirical studies that examine the challenges developers encounter by mining questions on SO.

3. Data collection

To gather posts from Stack Overflow (SO), we utilized the Stack Exchange Data Explorer web interface.¹⁴ This tool facilitates the retrieval of up-to-date SO posts based on specific criteria, such as tags related to our research interest. We opted not to use the SOTorrent dataset (Baltes et al., 2018), a commonly used SO data dump, because it has not been updated since December 2020. Fig. 2 provides an overview of our data collection process. Our data collection methodology, following previous studies (Ponzanelli et al., 2014; Rosen and Shihab, 2016), involves four steps:

Step 1: Identify Airflow-related tags. We manually compiled a list of tags containing the keyword 'airflow', including *airflow*, *airflow-scheduler*, *airflow-2.x*, *airflow-api*, *airflow-taskflow*, *airflow-webserver*, *airflow-connections*, *airflow-xcom*, *airflow-celery*, and *airflow-k8s*.

Step 2: Download Airflow-related question posts. Using the Stack Exchange Data Explorer, we retrieved all questions (totaling 9,737) tagged with any of the 10 identified tags. These posts date from September 03, 2015, to April 30, 2023, and include metadata such as the post identifier, type, creation date, title, body, tags, view count, score, favorite count, and the identifier of the accepted answer (for questions).

Step 3: Filter out negative questions. We eliminated questions with a negative score, resulting in a total of 9591 questions.

Step 4: Collect contextual information. For each retained question post from Step 3, we gathered its answers and all comments associated with both the questions and answers.

In the end, our dataset comprises 9591 SO questions, 10,386 answers, and 21,298 comments. This dataset supports our preliminary study and analytics for the three research questions (RQs). For the preliminary analysis, we utilized all questions (9591) and corresponding answers (10,386) from our dataset to examine the popularity and difficulty of Airflow-related questions. For RQ1 and RQ2, we

¹⁴ <https://data.stackexchange.com/stackoverflow/queries>

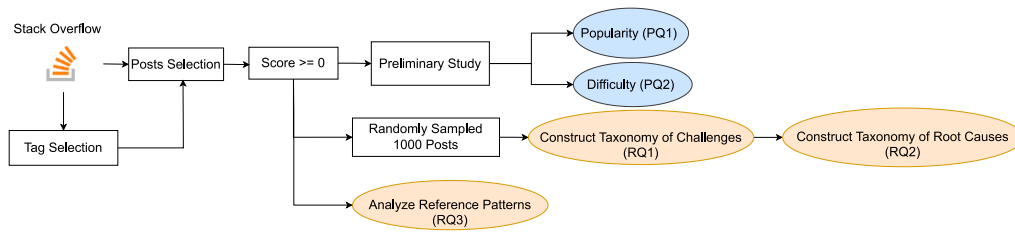


Fig. 2. Overview of our data collection process.

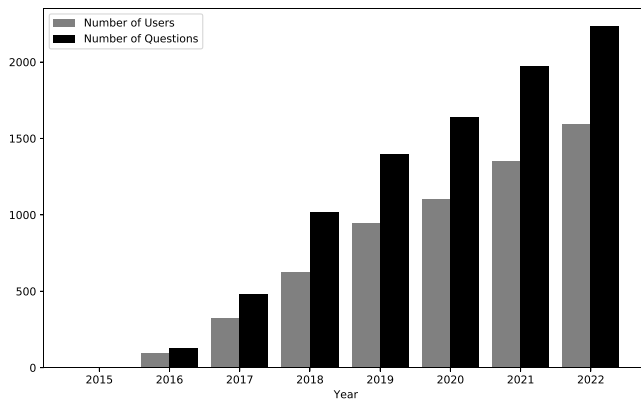


Fig. 3. The upward trajectory of the number of questions and users asking about Airflow over the years (calculated separately).

randomly sampled 1,000 question posts from our dataset, along with their answers. For RQ3, we developed a script to automatically extract all 16,059 shared URLs from the collected data, including questions, answers, and comments. The detailed approach for each RQ will be presented in the next sections.

4. Preliminary study

We conduct a preliminary quantitative analysis on the data (9591 questions and 10,386 answers) collected in Section 3. This preliminary study aims to answer two questions.

PQ1: What is the popularity of Airflow-related questions on Stack Overflow?

Motivation: Analyzing the popularity of Apache Airflow-related questions on SO is critical for understanding Airflow's prominence and impact within the developer community. Through a quantitative examination of Airflow-related questions, we can ascertain the degree of interest and engagement among developers and how it has evolved over the past years, which in turn reveals Airflow's traction and its potential establishment as a key tool in automated workflow development.

Approach: Following prior studies (Alshangiti et al., 2019; Chen et al., 2020), we assess the popularity of Airflow-related questions by examining both the volume of relevant questions on SO and the number of SO users posing these questions annually, spanning from 2015 to 2022. We exclude the year 2023 from this analysis, as our data collection only extends up to April of that year.

Result: Fig. 3 presents the annual trend in the popularity of Airflow-related questions, in terms of question number and involved user number, calculated separately. The result shows a significant upward trend in Airflow-related questions on SO from 2015 to 2022, signifying a steady escalation in user interest in Apache Airflow. Between 2019 and 2022, the annual increase in both users and questions on the platform ranged between approximately 16% and 18%. The results confirm the rising popularity and adoption of Airflow among developers.

PQ2: How difficult are Airflow-related questions?

Motivation: Airflow is challenging for newcomers to onboard (Yackel, 2021). In this question, we investigate the difficulty of answering Airflow-related questions, specifically by examining the time taken for these questions to receive a working solution. An answer to this question will not only quantify the complexity involved in resolving Airflow-related issues but also provide insights into the learning curve and potential knowledge gaps within the Airflow community.

Approach: Similar to prior SO studies in different domains (Yang et al., 2016; Rosen and Shihab, 2016; Ahmed and Bagherzadeh, 2018; Alshangiti et al., 2019; Bagherzadeh and Khatchadourian, 2019), we measure the difficulty of Airflow-related questions in two steps. We first collect Airflow-related questions with accepted answers and report the ratio of questions with accepted answers. For each selected question, we then calculate the duration between the posting time of the question and the accepted answer.

Result: Our data shows that only 36.4% (3491 out of 9591) of Airflow-related questions on Stack Overflow have accepted answers. As indicated in Table 1, this proportion is similar to that for deep learning development questions (37.3%), but it is lower compared to questions in the domains of big data, concurrency, and mobile development.

Regarding the median response time, it takes 527 min to receive an accepted answer for an Airflow-related question. This duration, as outlined in Table 1, is notably longer than the response times for other specialized domains explored. For example, mobile development questions typically receive an accepted answer within just 55 min. Table 2 details the distribution of accepted answers over various time frames within a subset of 3,478 posts. Notably, only 1.5% of accepted answers arrive within 5 min, 20.3% within an hour, and 85.7% within a week.

Summary of Preliminary Study

While the popularity of Airflow-related questions on Stack Overflow is on the rise, the significant median response time of 527 minutes for accepted answers suggests difficulties developers encounter in automated workflow development. This underlines the relevance and importance of analyzing the specific challenges faced by developers when using Airflow.

5. Empirical study and results

5.1. RQ1: What are the types of challenges developers face in using Airflow?

Motivation: The objective of RQ1 is to develop a taxonomy that summarizes the challenges developers encounter when implementing Workflows as Code with Airflow, as reflected in their questions on SO. This taxonomy offers a systematic framework for practitioners and researchers to understand and navigate the complexities associated with Airflow-related challenges. Additionally, by examining the prevalence of various challenge types, we aim to identify the most common issues newcomers should be aware of when employing Airflow in workflow development and management.

Table 1

Difficulty of Airflow-related questions versus other domains (ranked by median time to receive accepted answers in decreasing order). The most challenging domain for each metric is emphasized.

Domain	% of accepted answers	Median time to receive accepted answers (in minutes)
Airflow	36.4%	527
Deep Learning deployment, Deep Learning development (Chen et al., 2020)	29.3% , 37.3%	404.9, 145.8
Big Data (Bagherzadeh and Khatchadourian, 2019)	39.5%	198
Mobile (Rosen and Shihab, 2016)	45%	55
Concurrency (Ahmed and Bagherzadeh, 2018)	51.2%	42

Table 2

The percentage of questions (3478 with accepted answers) that receive an accepted answer within a specified time.

Accepted Answer within	Out of 3,478 questions
5 min	1.5%
1 h	20.3%
1 day	65.6%
7 days	85.7%
1 year	98.6%

5.1.1. Approach

We employed open coding (Seaman, 1999) for the manual categorization of challenge types, analyzing both the titles and bodies of selected Stack Overflow questions, including their embedded code snippets. Specifically, we randomly selected 1,000 questions from our dataset of 9591 questions (as detailed in Section 3). This sample size was chosen to ensure a 95% confidence level with approximately a 3% confidence interval, aligning with methodologies used in previous research (Wang et al., 2022). The categorization process was conducted in two stages and required 344 man-hours to complete:

- **Stage 1: Pilot construction.** Two co-authors began by independently categorizing 300 (30%) randomly chosen questions from the sample. They carefully reviewed each question together, labeling them based on the mentioned functionality of Airflow highlighted in the post. Similar labels were merged into broader categories; for instance, “*automated triggering*” and “*manual triggering*” were merged under “*scheduling*”. Questions not relevant or focused on Airflow (e.g., a developer is asking for a method to update the gcloud tool on Google Cloud Composer (GCP) worker nodes clarifying that it is a GCP issue rather than Airflow¹⁵) were classified as “*irrelevant*”. Following open coding, our taxonomy, with its categories and subcategories, was developed in a bottom-up approach, grouping subcategories into main categories based on their relevance to a primary functionality. This iterative process involved continuous refinement of the taxonomy through repeated analysis of the questions and labels, ensuring each question was assigned to the most fitting challenge category.
- **Stage 2: Extension to the full set.** Next, the first two authors independently applied the established taxonomy from Stage 1 to categorize the remaining 700 questions. The annotators achieved a Cohen’s kappa coefficient of 0.81, indicating substantial inter-rater agreement (Viera et al., 2005; Landis and Koch, 1977). They then discussed and resolved conflicts for the 700 questions, finalizing the taxonomy.

5.1.2. Results

We observe that 58 out of 1000 (5.8%) labeled questions are *irrelevant* to Airflow. For the remaining questions, Fig. 4 presents an overview of our taxonomy, which is composed of 14 fine-grained subcategories and 7 high-level categories. Below, we describe each category in more detail.

(C1) Workflow Definition: Our analysis reveals that 36.6% of the Airflow-related questions seek guidance in defining workflows. This finding aligns with expectations. Enabling “*Workflows as Code*” is the core feature of Airflow. This paradigm, while powerful, presents a learning curve for developers, especially in comprehending how to translate complex workflow logic into executable code effectively.

This high-level category is encompassed by the following four subcategories, ordered by their prevalence:

(C1.1) Task Definition and Configuration: The most frequently encountered type of question in Category C1 asks about the defining and configuring tasks, accounting for 20.1% of the Airflow-related questions. Tasks in Airflow are mostly defined using operators, which are specialized classes that encapsulate the logic for a specific task. Airflow offers a range of general operators like BashOperator and PythonOperator for executing a bash command or calling an arbitrary Python function, as well as provider-specific operators such as BigQueryUpdateTableOperator and AWS AthenaOperator, enabling integration with various services. Configuring a task thus involves selecting an appropriate operator and carefully setting a multitude of parameters associated with it, which can be a complex process for developers. For instance, developers often struggle with configuring the BashOperator, particularly when needing to set the bash_command argument with dynamic values extracted from intricate JSON structures.¹⁶

In addition to using built-in operators, the creation of custom operators in Airflow is frequently necessary for workflows with specific demands. Yet, mimicking the comprehensive functionality of the built-in operators can be challenging. For example, a developer creating a custom operator might face difficulties with accessing the task_instance object, a feature readily available in standard operators like PythonOperator, but not directly accessible in custom ones.¹⁷

(C1.2) Integration with External Systems: Our result reveals that 9.6% of the Airflow-related questions ask about issues when integrating external systems, such as data storage service and data processing frameworks, with Airflow. As a workflow orchestration tool, Airflow’s ability to seamlessly interact with these external systems is crucial for the successful execution of tasks and workflows. Unfortunately, we observe that developers frequently face challenges in such integration process, starting from the very first step, i.e., establishing the connections between Airflow and external systems. For example, a

¹⁵ <https://stackoverflow.com/questions/63965159>

¹⁶ <https://stackoverflow.com/questions/74444283>

¹⁷ <https://stackoverflow.com/questions/59325584>

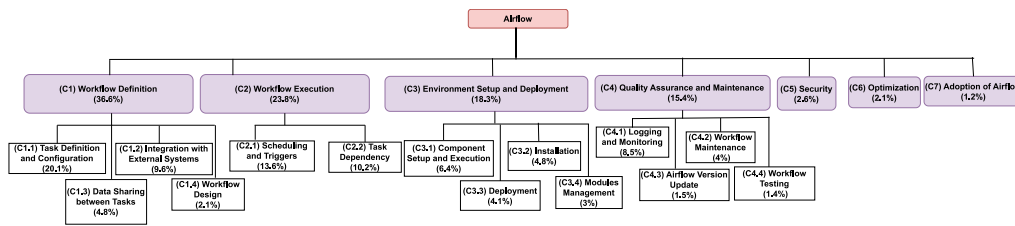


Fig. 4. A hierarchical taxonomy of challenges faced by developers in utilizing Airflow.

developer had trouble while implementing a task using the `EmrCreateJobFlowOperator` in Airflow and is getting an error about an undefined connection ID (`emr_default`).¹⁸ This error suggests that the required connection information for Amazon EMR (a cloud big data platform) is missing. Besides connection issues, developers also encounter difficulties in the configuration of the external systems.

(C1.3) Data Sharing between Tasks: We observe that 4.8% of the Airflow-related questions concerns about the exchange of data — information, results, or artifacts between various tasks within a workflow. Effective data sharing is crucial to the orchestration of complex data processing pipelines, yet developers often encounter obstacles in efficiently accessing or storing data between tasks. For example, one developer faced difficulties in devising a method to pass data to a subsequent task.¹⁹ The task involved fetching data from ten REST API endpoints and saving each dataset into a data lake. The primary challenge for the developer was finding an optimal way to transfer large datasets from the data-fetching task to the data-saving task. The Airflow built-in method for data sharing, `XCom`,²⁰ was not feasible due to its limitations on data size, motivating the developer to seek alternative solutions.

(C1.4) Workflow Design: In 2.1% of the Airflow-related questions, developers did not ask specific technical details, but rather how to optimally design workflows tailored to specific scenarios. This is particularly challenging when workflows require intricate interactions between various services or systems. For instance, a developer asked how to design a workflow that downloads tables from a QA Redshift (an Amazon-managed data warehouse service) to an S3 bucket and then transfers them to another Redshift instance, using `LocalStack` (a mocked AWS service).²¹

(C2) Workflow Execution: Our analysis reveals that 23.8% of the Airflow-related questions seek guidance in defining workflow execution order and addressing runtime issues during workflow execution. These issues can be intricate, stemming from diverse factors such as data dependencies, scheduling conflicts, or resource constraints.

(C2.1) Scheduling and Triggers: A significant portion of Airflow-related queries, representing 13.6% of the total, focuses on scheduling workflows and configuring triggers. These scheduling issues often originate from inaccuracies in defining schedule intervals or handling time zone variances. An example includes a developer encountering difficulties in precisely timing workflow executions.²² In Airflow, triggers are conditions or events typically defined within a DAG (as shown in Lines 5 and 6 of Fig. 1), enabling the automatic initiation of tasks under certain conditions. However, there are scenarios where developers need to trigger DAG, which runs externally and is facilitated through an HTTP request to the `dagRuns` endpoint of the Airflow REST API. This process can be challenging and error-prone, as illustrated by a specific

case.²³ These findings reveal the need for more intuitive and user-friendly methods for scheduling and triggering in Airflow to enhance both automated and external execution processes.

(C2.2) Task Dependency: 10.2% of Airflow-related questions is centered around task dependency issues. Task dependencies are the directed edges that determine how to move through the DAG, which defines a workflow. Properly managing task dependencies is crucial for ensuring that tasks are executed in the correct order. However, we observe that developers face difficulties in defining those dependencies, especially when encountering dynamic tasks (generated at runtime based on current data). For example, a developer was facing difficulties while trying to configure dependencies dynamically from tasks and their dependencies defined in a data frame.²⁴

Moreover, effective task dependency management in Airflow extends beyond defining task relationships, and it also involves adapting to status changes during runtime. For example, if an upstream task fails to find files in an S3 (Amazon's Storage Service) bucket, it should trigger an email notification and stop a subsequent task, instead of executing the subsequent task. Developers often seek solutions for implementing such conditional execution flows within their workflows.²⁵

(C3) Environment Setup and Deployment: Our analysis reveals that developers often encounter challenges when setting up Airflow in their development environment and deploying their DAGs in production, representing 18.3% of all Airflow-related questions. This high-level category is encompassed by the following four sub-categories, ordered by their popularity.

(C3.1) Component Setup and Execution: As described in Section 2, Airflow is composed of four key architectural components: schedulers, a web server, a folder of DAG files, and a metadata database. For a successful launch of a DAG, these components (e.g., a MySQL database to serve as the metadata database) must be properly set up and correctly configured. Nonetheless, developers often encounter challenges in this setup process, as well as in executing those components. For instance, a developer encountered a “disk I/O error” while attempting to run the “`airflow scheduler`” and “`airflow webserver`” commands in Airflow.²⁶

(C3.2) Installation: Our analysis shows that 4.8% of Airflow-related questions report issues developers faced in the installation of Airflow. For example, a developer encountered a problem while attempting to install Airflow on a very specific environment, i.e., macOS Catalina with Python 3.8.2, using the “`pip install -airflow`” command.²⁷

(C3.3) Deployment: We observe that 4.1% of the Airflow-related questions are concerns about deploying Airflow from the development environment to the production environment. The primary reason for these concerns is the complexity involved in transitioning between different environments. This includes ensuring consistency in configurations, managing dependencies, and handling differences in infrastructure setups. Additionally, adapting workflows to the constraints

¹⁸ <https://stackoverflow.com/questions/70222696>

¹⁹ <https://stackoverflow.com/questions/69487557>

²⁰ <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/xcoms.html>

²¹ <https://stackoverflow.com/questions/71504505>

²² <https://stackoverflow.com/questions/70228718>

²³ <https://stackoverflow.com/questions/67110383>

²⁴ <https://stackoverflow.com/questions/75166872>

²⁵ <https://stackoverflow.com/questions/68114312>

²⁶ <https://stackoverflow.com/questions/73311291>

²⁷ <https://stackoverflow.com/questions/64832243>

and requirements of production environments, such as security protocols and resource limitations, can be a demanding task. These factors often require developers to make significant adjustments that are error-prone. For example, a developer experienced a deployment-related issue while trying to deploy Airflow on Azure Kubernetes Services (AKS) and load DAGs from a GitHub repository using Helm charts.²⁸

(C3.4) Module Management: The least prevalent type of questions in C2 is concerned with the module management feature offered by Airflow, accounting for 3% of Airflow-related questions. Airflow enables developers to import Python modules in DAG files for defining and executing tasks. These modules can encompass custom functions, classes, or additional code essential for task execution. However, configuring these dependent modules correctly could be challenging for developers. For instance, a developer experienced a `ModuleNotFoundError` when attempting to import a Python method from a file located in a local directory.

(C4) Quality Assurance and Maintenance: According to our analysis, 15.4% of the Airflow-related questions concern the quality and ongoing maintenance of workflows managed through the platform. This high-level category contains the following four sub-categories, ordered by their popularity.

(C4.1) Logging and Monitoring: In 8.5% of the Airflow-related questions, developers encountered challenges with Airflow's logging and monitoring capabilities. Given that workflows typically operate autonomously, effective observability through logging is crucial. Apache Airflow's logging framework is designed to offer comprehensive insights into task and DAG executions. While its default setup writes logs to the local file system—adequate for development and debugging phases—production environments, particularly in cloud settings, often necessitate remote logging capabilities to services like AWS S3, Google Cloud Storage, and Azure Blob Storage. However, developers frequently encounter hurdles in both local and remote logging setups. An example case is a developer who faced issues with the Airflow UI redirecting to localhost instead of the EC2 instance's IP address when trying to access task logs, requiring manual intervention to correct the URL.²⁹ Another developer struggled with configuring Airflow to write logs to an S3 bucket.³⁰

(C4.2) Workflow Maintenance: Our analysis reveals that 4% of the Airflow-related questions are related to challenges in workflow maintenance. The maintenance of workflows in Airflow presents several difficulties. One of the primary challenges is managing the dependencies and versioning of complex DAGs along with their associated libraries, particularly in the absence of a dedicated versioning mechanism for workflows within Airflow. An example from our annotated questions includes a developer seeking guidance on how to implement DAG versioning in Airflow, aiming to enhance the efficiency of management and maintenance processes.³¹ Furthermore, as the number and complexity of tasks and workflows expand, scaling Airflow to adequately handle an increasing workload and meet escalating resource requirements emerges as another type of complexity.

(C4.3) Airflow Version Update: In some questions (1.5% of the Airflow-related questions), developers reported issues faced when updating to newer versions of Airflow. Even with the support features provided by Airflow, such as automated methods for upgrading the metadata database, migration issues still occur. For instance, an error was found while a developer was migrating from Airflow version 1.10.15 to 2.2.5.³² After the update, the developer encountered a problem when using the `BigQueryToGCSOperator` to transfer data from BigQuery to GCS. The process resulted in a `NotFound` exception

and a 404 error, indicating that the job linked to the operator could not be found – a problem that did not exist in Airflow 1.10.15. The developer sought to understand the root cause of this compatibility issue and explore possible solutions for smooth migration.

(C4.4) Workflow Testing: Our analysis shows that developers sometimes (1.4% of the Airflow-related questions) ask questions about testing of tasks and DAG in Airflow. Effective testing is essential for ensuring the reliability of workflows. This encompasses unit testing to evaluate individual components of a DAG, DAG testing to confirm the overall structure and integrity of the workflow, and integration testing to examine how the workflow interacts with external systems. Despite its importance, developers encounter challenges in successfully conducting these tests within the Airflow framework. For example, a developer faced difficulties while carrying out unit tests for an ETL process in Airflow, particularly aiming to ensure that their DAGs were free of cycles.³³

(C5) Security: Our analysis reveals that 2.6% of the Airflow-related questions focus on security implementation challenges within the platform. Airflow has a security model ensuring the integrity and confidentiality of the tasks being executed. The security model adopts robust authentication mechanisms, such as LDAP and OAuth. Despite these foundational security designs, developers often encounter complexities in setting up and managing these advanced security protocols. For instance, a developer encountered issues in the implementation of LDAP authentication in Airflow.³⁴ The specific challenge lies in safely managing the LDAP bind password within Airflow's configuration settings.

(C6) Optimization: Our analysis indicates that 2.1% of the Airflow-related questions involves developers seeking advice on optimizing their Airflow configurations. Optimization in this context refers to enhancing the system's efficiency, responsiveness, and resource management to ensure workflows are executed both timely and smoothly. A common optimization challenge is to address excessive CPU usage in Airflow setups. For example, one user reported a high CPU utilization, up to 20%, on an EC2 instance running Airflow with Docker Compose and LocalExecutor, even when the system was idle.³⁵ The user sought advice on configuration adjustments to lower CPU usage, as well as insights into the potential trade-offs of such changes.

(C7) Adoption of Airflow: We find that in 1.2% of the Airflow-related questions, developers seek decision-making guidance on whether to adopt Airflow given their technical stack and requirements.

Summary of RQ1

We identify a comprehensive hierarchical taxonomy for Airflow-related challenges faced by developers, encompassing 7 high-level categories and 14 subcategories. The most prevalent among these high-level categories encompasses questions that seek guidance in defining workflows and addressing issues that arise during workflow execution. Developers also face other difficulties, such as setting up deployment and production environments for Airflow and ensuring the quality of workflows.

5.2. What are the root causes of the challenges?

Motivation: In RQ1, we identified a wide spectrum of questions developers encounter when utilizing Airflow for their workflow development and management. However, similar types of questions can stem from different root causes, and conversely, diverse questions may originate

²⁸ <https://stackoverflow.com/questions/71258379>

²⁹ <https://stackoverflow.com/questions/49784569>

³⁰ <https://stackoverflow.com/questions/63992194>

³¹ <https://stackoverflow.com/questions/61796692>

³² <https://stackoverflow.com/questions/72529432>

³³ <https://stackoverflow.com/questions/67725703>

³⁴ <https://stackoverflow.com/questions/71249044>

³⁵ <https://stackoverflow.com/questions/67063149>

from the same fundamental issue. Therefore, in RQ2, we perform a manual analysis to identify the underlying root causes of the Airflow questions. By uncovering these root causes, our research can provide practitioners with valuable knowledge, allowing them to navigate common issues more effectively. Furthermore, the results can inspire future research into creating more efficient support mechanisms and tools for developers working with Airflow, thereby enhancing the overall user experience and effectiveness of the platform.

5.2.1. Approach

We carried out a manual analysis on the same set of 1000 questions previously sampled for RQ1, employing a similar two-stage open coding methodology. The coding process was completed over the course of approximately 450 h.

- **Stage 1: Pilot construction.** In this stage, two co-authors collaboratively established the root causes for a pilot set of 300 posts. The 21 question posts previously categorized as “irrelevant” to Airflow during RQ1 were excluded from this analysis. To identify the root cause of each question, the annotators carefully examined each element of a question post, including its title, body text, code snippets, associated answers, comments, and even referenced URLs. This comprehensive review ensured that the root cause was identified based on its detailed explanation in the discussion. If the specific root cause for a question remained ambiguous, it was labeled as “unclear”. Specifically, 50 question posts were labeled as “unclear”. Through this process, the annotators developed a taxonomy encompassing 10 distinct root causes.
- **Stage 2: Extension to the full set.** Next, the two annotators individually applied the taxonomy established in Stage 1 to the remaining 700 questions, excluding 37 questions that are irrelevant to Airflow. The inter-rater agreement during the independent labeling is 0.79, measured by Cohen’s Kappa (Viera et al., 2005), indicating substantial inter-rater agreement. The annotators then discussed and resolved all conflicts and finalized the taxonomy. In this stage, 128 question posts were labeled as “unclear”.

5.2.2. Results

Table 3 presents the taxonomy of root causes we identified through our manual analysis of 942 Airflow-related questions (as mentioned in RQ1, 58 out of the 1000 sample questions are not relevant to Airflow). The findings indicate that the three most prevalent root causes are incorrect workflow configurations in DAGs, challenges with complex environment configuration, and a lack of basic knowledge about Airflow (e.g., concepts related to DAG). Combined, these three categories account for 46.7% of all Airflow-related questions. Notably, 6.5% of the challenges stem not directly from Airflow itself but from a deficiency in understanding the external systems with which Airflow interacts. Below, we describe each category in more detail.

(R1) Incorrect Workflow Configuration: Misconfigurations in DAGs account for 18.9% of the Airflow-related issues we analyzed. Such configuration challenges can arise at both the task and DAG levels, involving aspects like setting operator parameters for tasks or scheduling workflows. An example is a developer failing to correctly schedule a workflow involving a crucial but optional parameter named `execution_timeout`.³⁶

We further investigated potential factors that may contribute to the prevalence of this category and found three. First and foremost, Airflow’s documentation does not provide a centralized reference for all mandatory and optional parameters, leaving developers without clear guidance on their usage. Secondly, while Airflow allows for dynamic parameter configuration at runtime for tasks and DAGs, it lacks clarity

on which parameters are compatible with this feature. Thus, developers often fail to configure dynamic parameters in their DAGs. Last but not least, Airflow’s official documentation often omits critical information regarding the interdependencies and constraints of multiple parameters (e.g., two parameters should be configured together), leading to confusion and misconfiguration. For instance, understanding the interplay between scheduling parameters i.e., `schedule_interval` and `start_date` is needed to ensure the desired execution behavior of the DAG.³⁷

(R2) Complex Environmental Configuration: We observe that 14% of Airflow-related questions are rooted in complex environmental and component configuration. Airflow is recognized for its versatile configuration options, supporting a wide array of infrastructures and use cases. These configurations are primarily managed via the “`airflow.cfg`” file, which offers extensive customization of settings. However, the platform’s diversity in configurations can lead to significant challenges. For instance, one developer struggled with setting up the metadata database.³⁸

Challenges also arise when integrating Airflow with distributed task queueing systems like CeleryExecutor, especially when managing data storage access across multiple worker nodes.³⁹

In containerized environments like Docker, or orchestration platforms like Kubernetes, configuring Airflow involves a deep understanding of these platforms’ specific features, often going beyond the `airflow.cfg` file to include Docker and Kubernetes settings. For instance, in Docker, the Dockerfile serves as the blueprint for building the Airflow image, which entails various steps: selecting a base image, installing dependencies, configuring Airflow, and adding DAGs and plugins. While Airflow provides a standard Dockerfile, it frequently requires customization. For instance, a developer encountered issues installing Python packages in a virtual environment due to a conflict with the `PIP_USER` environment variable set in the original Dockerfile.⁴⁰ The default setting activated the `—user` flag, which was incompatible with the developer’s attempts to install packages in the virtual environment, resulting in errors.

Configuration through the web interface can also initiate challenges. For example, despite configuring the connection details in Airflow’s web interface, a developer encounters difficulties in making the connection work from within the Airflow container.⁴¹

(R3) Insufficient Basic Knowledge of Airflow: Our analysis reveals that 13% of the studied problems are related to a lack of basic knowledge of Airflow. Developing effective workflows in Airflow demands a solid grasp of its core concepts, including Directed Acyclic Graphs (DAGs), basic operators, task dependencies, and the Airflow Command Line Interface (CLI). Misunderstandings of these foundational aspects can lead to challenges.

An example involves a developer who faced difficulties with DAGs not appearing in the web UI when defined in separate files.⁴² This issue stems from a misunderstanding of Airflow’s DAG discovery mechanism, which relies on specific conventions to identify DAGs in Python files if DAGs are defined in multiple files. The developer’s approach did not align with these conventions. To address this, another developer suggested including the term “`airflow`” in the file, even as a comment, to meet the discovery criteria. This issue exemplifies a common knowledge gap regarding the DAG discovery conventions in Airflow.

(R4) Limitations of Airflow: 8.2% of the questions related to Airflow arise from inherent limitations in specific versions, such as bugs or unsupported features. These issues can significantly disrupt development,

³⁷ <https://stackoverflow.com/questions/74235924>

³⁸ <https://stackoverflow.com/questions/70903197>

³⁹ <https://stackoverflow.com/questions/66750928>

⁴⁰ <https://stackoverflow.com/questions/73962053>

⁴¹ <https://stackoverflow.com/questions/70267145>

⁴² <https://stackoverflow.com/questions/70051020>

³⁶ <https://stackoverflow.com/questions/54810074>

Table 3

Root causes derived from the manual analysis of 942 Airflow-related posts in Stack Overflow.

Root cause	Definition	% of Questions
R1:Incorrect Workflow Configuration	Developers face many issues within Airflow implementations due to the omission or misconfiguration of settings related to workflow configuration at the task level and DAG level.	18.9
R2:Complex Environmental Configuration	The wide range of configuration options and settings required for Airflow, can pose challenges in ensuring optimal setup and performance.	14
R3:Insufficient Basic Knowledge of Airflow	Developers need to have a basic understanding of Airflow's core concepts, including Directed Acyclic Graphs (DAGs), operators, and task dependencies, to effectively design, implement, and manage workflows.	13
R4:Limitations of Airflow	Developers encounter challenges due to issues or defects, or unsupported features that require extra effort within the Airflow platform.	8.2
R5:Insufficient Knowledge of the External Systems	Lack of knowledge about external systems/domains, including cloud storage, database systems, and managed Airflow services, can lead to several challenges.	6.5
R6:Missing or Incorrect Dependency	When working with Airflow, developers often encounter challenges due to missing dependencies, which can complicate both the setup of the necessary environment and the definition of their tasks.	5.7
R7:Lack of knowledge in Programming and Devops	Developers encounter challenges due to knowledge gaps in Python and a lack of understanding of the development and DevOps process while working with Airflow.	5.6
R8:Incorrect or Sub-optimal Operator Choice	Airflow provides a wide range of operators, each designed for different use cases and functionality. The selection of an operator that does not align well with the specific requirements of a task or job within a workflow. (Misuse/Unknown to operator)	4.8
R9:Complex Workflow	Complex workflows, with their intricate processes, dependencies and dynamic behavior present challenges for developers, making it difficult to manage and understand them effectively.	3.1
R10:Breaking Changes in Airflow	Developers may use an outdated API that is no longer compatible with a newer version of Airflow.	0.8
Unclear	Lack of information to derive a root cause.	19.3

often requiring developers to devise workarounds or participate in the community's problem-solving efforts. For instance, a developer using Airflow 2.0.0 encountered an unexpected behavior where a workflow set for daily execution skipped a day in its schedule, despite being correctly set up and manually triggered.⁴³ This issue was traced back to a bug in the Airflow 2.0.0 release,⁴⁴ which was subsequently fixed in the 2.0.1 update. In another case, a developer faced challenges in executing a dynamic number of tasks, with a maximum of four tasks in parallel.⁴⁵ This led to the introduction of a new feature in Airflow to better support dynamic task execution.⁴⁶ Issues have also been reported with Airflow's Role-Based Access Control (RBAC) system, especially regarding its limited functionality when applied to developer actions through the Experimental API or the Airflow CLI.⁴⁷

It is worth noting that bugs may occur not only in Airflow's core framework but also in provider-specific operators. For example, a bug introduced in version v7.0.0 of the Airflow Providers Google package could lead to task failures, necessitating workarounds such as reverting to an earlier version or using the BigQuery API directly.

(R5) Insufficient Knowledge of the External Systems: Our analysis shows the root cause of 6.5% of the studied problems is related to insufficient knowledge about external systems that Airflow interacts with

through its tasks. For instance, a developer using the `GoogleCloudStorageComposeOperator` in Google Cloud Composer encountered the HTTP 429 Too Many Requests error.⁴⁸ This error suggests the rate limit was exceeded for changes to the "object path/file.csv", likely due to Google Cloud Storage's write limit of once per second for the same object name. The issue indicates that the developer was unfamiliar with or overlooked these specific rate limits - a common concept that needs to be aware of when using Google Cloud Storage. In another scenario, a developer working with Amazon Managed Workflows for Airflow (MWAA) faced challenges as the platform restricted the installation of additional packages on the webserver,⁴⁹ which can be resolved if the developer knows how to establish connections to Google Cloud through the webserver.

(R6) Incorrect or Sub-optimal Operator Choice: Our analysis reveals that 5.7% of the Airflow-related issues stem from developers making incorrect or sub-optimal choices of operators for their tasks. Airflow offers a range of built-in operators for basic tasks and provider-specific operators for more specialized tasks interacting with external systems. Each operator in Airflow is tailored for specific use cases, and choosing the wrong one can lead to inefficiencies or added complexities in the workflow. For example, a developer used the `PythonOperator` for tasks where the `PythonVirtualenvOperator` might be a more optimal choice, particularly for handling Python dependencies in a virtual environment.⁵⁰

⁴³ <https://stackoverflow.com/questions/65897976>

⁴⁴ <https://github.com/apache/airflow/issues/13434>

⁴⁵ <https://stackoverflow.com/questions/75192266>

⁴⁶ <https://github.com/apache/airflow/issues/29084>

⁴⁷ <https://stackoverflow.com/questions/62759527>

⁴⁸ <https://stackoverflow.com/questions/69988105>

⁴⁹ <https://stackoverflow.com/questions/68404064>

⁵⁰ <https://stackoverflow.com/questions/67615888>

Furthermore, navigating the broad selection of built-in and provider-specific operators can be challenging. In one instance, a developer sought advice on choosing the right operator to load a file from Google Cloud Storage (GCS) into BigQuery, knowing the operator for the reverse process but unclear about the best choice for this particular task.⁵¹ The difficulty here lies in selecting from a diverse array of available operators,⁵² each suited to different aspects of interacting with GCS and BigQuery.

(R7) Lack of knowledge in Programming and DevOps: Our analysis indicates that 5.6% of Airflow-related questions are related to developers' limited knowledge in programming, specifically Python, and DevOps practices. Since Airflow workflows are defined through Python code, proficiency in Python is essential. Additionally, Airflow's capabilities in logging and monitoring workflows are crucial for ensuring their reliability during runtime, necessitating a comprehensive understanding of the software development lifecycle, including practices like test-driven development and DevOps methodologies. Airflow users, however, come from diverse backgrounds and vary in their experience levels, which can lead to challenges. A case in point involves a developer who faced difficulties with an Airflow task intended to consume messages from a Kafka topic via the `ConsumeFromTopicOperator`.⁵³ Despite the DAG executing without any visible errors, the anticipated messages were not reflected in the Airflow logs. This issue was ultimately attributed to a lack of understanding in Python programming, especially in areas such as variable scopes and the application of callable functions.

(R8) Missing or Incorrect Dependency: The root cause of 4.8% of the studied problems is related to Python packages or external libraries that are either not installed or improperly configured. Such issues can lead to errors and warnings that impede the system's functionality. For instance, a developer encountered a `No module named boto3` error in Airflow even after installing the `boto3` library via `pip`.⁵⁴ This error indicates that the `boto3` library was not accessible in the Python environment where Airflow's tasks were running. To resolve such issues, the `PYTHONPATH` environment variable within Airflow must be correctly set to include the directories where the necessary libraries are installed. In environments with distributed architectures, where workflows extend across multiple nodes or clusters, managing environmental dependencies is especially crucial. Each node involved in the workflow must have the appropriate library versions installed to avoid conflicts and ensure a consistent and efficient workflow.

(R9) Complex Workflow: The root cause in 3.1% of studied problems is related to the inherent complexity of the workflows being developed. These workflows often consist of multiple interconnected tasks with complex dependencies, demanding advanced and thoughtful workflow design. For instance, a developer faced challenges in accurately defining task relationships within a complex workflow that required both sequential and parallel execution patterns.⁵⁵ The specific DAG in question initially involved sequential tasks (`step_1` and `step_2`), followed by the parallel execution of three tasks (`X`, `Y`, `Z`). The complexity was raised when the developer needed to integrate additional sequential dependencies (tasks `A` and `B`) within the parallel segment of the workflow, specifically for task `X`, while maintaining the parallel execution of tasks `Y` and `Z`.

(R10) Breaking Changes in Airflow: 0.8% of the Airflow-related questions we analyzed are related to breaking changes introduced in newer versions of Airflow. As Airflow continues to evolve, its APIs undergo changes, including the introduction of new ones or modifications to existing ones. Developers who are using older versions of

Airflow might encounter compatibility issues if they rely on outdated APIs. For example, a developer encountered an error in Airflow 2.5.1 due to the absence of the `"execution_date"` attribute in the `TaskFail` object.⁵⁶ This problem stemmed from a change made in Airflow 2.3, which deprecated the `"execution_date"` attribute in the `TaskFail` class. However, the removal of this attribute was not clearly communicated in the error messages or the changelog for Airflow 2.3, and this breaking change took effect in version 2.5.1, causing the error.

Unclear: The root cause for the 19.3% of the Airflow-related questions was labeled as unclear because these questions either lack accepted answers or do not contain informative discussions or clarifications in their question description, answers, and comments.

Summary of RQ2

We identify 10 types of root causes underlying the challenges that developers encounter with Airflow, with incorrect configurations in DAGs emerging as the most common issue. These root causes indicate a dual problem: firstly, there are notable gaps in Airflow's official documentation that fail to adequately guide users; secondly, there is a shortfall in developers' understanding and knowledge in terms of Airflow, external systems that interact with Airflow, general programming, and DevOps.

5.3. What are the types of online sources developers refer to in Airflow-related posts?

Motivation: In RQ2, we identified a notable knowledge gap between Airflow's documentation and the needs of developers, which contributes significantly to the challenges faced in Airflow-related projects. Building on this insight, RQ3 aims to explore the common types of shared resources and websites in questions related to Airflow. Our analysis results can shed light on how external information is utilized by the community to address specific challenges and questions related to Airflow.

5.3.1. Approach

To address RQ3, we implemented the following methodology:

- **Step 1: Collect shared URLs.** Our first step involved extracting URLs from all components (questions, answers, comments) of all the collected 9737 SO questions with Airflow-related tags. We utilized regular expressions to capture the URLs and then counted the frequency of URL sharing across questions, answers, and comments.
- **Step 2: Identify root domains.** We then automatically extracted the root domains of these URLs to understand where these resources were hosted. After analyzing these domains, we ranked them based on their frequency of occurrence to determine the most commonly referenced sources. Note that we do not consider image resources. Image hosting sites like `"i.stack.imgur.com"` were ignored.
- **Step 3: Merge similar domains.** To further assess the relevance of these popular resources to Airflow, we focused on the Top-20 most commonly cited domains from Step 2, which collectively accounted for 85% of all shared URLs. Our review revealed a variety of domains with high relevance, such as `github.com` and `gist.github.com`, known for hosting source code. We then consolidated similar domains to eliminate duplicates. Furthermore, we manually merged several domains based on their specific relevance to Airflow. For example, domains that provide documentation for external systems with which Airflow interacts were

⁵¹ <https://stackoverflow.com/questions/74191180>

⁵² <https://airflow.apache.org/docs/apache-airflow-providers-google/stable/operators/cloud/bigquery.html>

⁵³ <https://stackoverflow.com/questions/74109768>

⁵⁴ <https://stackoverflow.com/questions/61562079>

⁵⁵ <https://stackoverflow.com/questions/71052585>

⁵⁶ <https://stackoverflow.com/questions/75612221>

Table 4
Prevalence of the URLs in questions, answers, and comments.

Type	URLs Occurrences	Associated Questions (% in Airflow-related Questions)
Comments	3,434	2,268 (23.65%)
Questions	4,533	2,828 (29%)
Answers	9,092	4,389 (45.76%)

Table 5
A taxonomy of top-20 most popular resources referred to in Airflow-related question posts.

Category	% of URLs	Domains
Documentation (Airflow)	27.55%	airflow.apache.org
	1.1%	airflow.incubator.apache.org
GitHub	24.33%	github.com
	0.34%	gist.github.com
Question and Answer site	14%	stackoverflow.com
Documentation (External Systems)	8.76%	cloud.google.com
	1.14%	docs.aws.amazon.com
	0.46%	kubernetes.io
	0.75%	docs.docker.com
	0.32%	aws.amazon.com
Content Sharing	1.40%	medium.com
	1.1%	towardsdatascience.com
	0.36%	youtube.com
Issue Tracking	1.11%	issues.apache.org
Third Party Documentation	1.82%	docs.astronomer.io
	0.34%	www.astronomer.io
Package Management	0.81%	pypi.org
Collaboration Wiki	0.65%	cwiki.apache.org
Documentation (Python)	0.59%	docs.python.org
Editor for Cron Scheduling	0.32%	crontab.guru

grouped into one category. Last, we name each category based on its nature and relevance to Airflow.

5.3.2. Results

Our analysis shows that external resources are frequently referred to in Airflow-related questions on Stack Overflow. We identified a total of 16,059 instances of URL sharing across questions, answers, and comments. Notably, 69.53% of all Airflow-related question posts contain at least one external URL reference. As shown in Table 4, most URLs occur within answers, highlighting that answers often rely on external resources to provide comprehensive solutions or further information.

Table 5 shows the Top-20 most frequently referenced domains in Airflow-related discussions, organized into a taxonomy of 11 distinct categories. The analysis reveals that a significant 38.13% of these domains are centered around documentation. This documentation spans a broad spectrum, encompassing Airflow itself, external systems, those reorganized by third-party, and Python-related resources. In addition to documentation, the results indicate a high frequency of references to GitHub and various Q&A sites.

Our analysis yields several key observations. First, Airflow's official website, airflow.apache.org, is the most frequently cited source, accounting for 27.55% of all URLs. This site primarily offers basic information and functionalities about Airflow. However, documentation of external systems, particularly Google Cloud services found on cloud.google.com, is also notably prevalent, comprising 8.76% of the references. This highlights the importance of understanding both Airflow and its integrations with external cloud services.

The second most referenced source is github.com. Our further quantitative analysis of this domain finds that among all URLs from the GitHub domain, a significant 70.3% are associated with the Airflow repository. Within this specific group, a quarter (25%) of the URLs

point to issues or pull requests of Airflow. A further qualitative examination reveals that GitHub links are often shared to directly reference the source code and help understand how a particular feature or function is implemented that can guide to resolving an issue. Additionally, sharing links to issues and pull requests on GitHub helps identify specific problems or feature requests, facilitating collaboration on solutions and engaging the community in discussions about project direction and improvements.

The website [Astronomer.io](https://astronomer.io) contributes to 1.33% of the shared URLs. Astronomer.io provides documentation for Astro, a cloud solution designed to enhance the management of Apache Airflow by adding extra functionalities. This suggests a trend in the community towards leveraging third-party documentation sources for extended knowledge and capabilities beyond what is available in the core Airflow documentation.

Summary of RQ3

In Airflow-related questions on Stack Overflow, external resources are prominent, particularly in the answers. The most frequently referenced resource is Airflow's official website, indicating its central role as a primary source of information. However, our analysis also shows a significant reliance on documentation from external systems and third-party resources. This trend indicates the diverse range of information sources developers consult to address Airflow-related challenges. Moreover, the developers often turn to the official GitHub repository of Airflow for deeper and more detailed insights and solutions, highlighting its significance within the Airflow community.

Table 6

The Top-4 most prevalent root causes associated with the seven high-level Airflow-related challenges.

Challenges	Root Cause	Percentage (%)
(C1) Workflow Definition	Incorrect Workflow Configuration	36.24
	Insufficient Basic Knowledge of Airflow	13.76
	Insufficient Knowledge of the External Systems	10.40
	Limitations of Airflow	8.72
(C2) Workflow Execution	Incorrect Workflow Configuration	35.33
	Insufficient Basic Knowledge of Airflow	23.95
	Limitations of Airflow	11.98
	Complex Workflow	10.18
(C3) Environment Setup and Deployment	Complex Environmental Configuration	44.06
	Missing or Incorrect Dependency	25.87
	Lack of knowledge in Programming and Devops	10.49
	Insufficient Basic Knowledge of Airflow	8.39
(C4) Quality Assurance and Workflow Maintenance	Complex Environmental Configuration	23.85
	Limitations of Airflow	20.18
	Insufficient Basic Knowledge of Airflow	15.60
	Lack of knowledge in Programming and Devops	10.39
(C5) Security	Complex Environmental Configuration	37.50
	Insufficient Knowledge of the External Systems	25.00
	Limitations of Airflow	23.53
	Lack of knowledge in Programming and Devops	6.25
(C6) Optimization	Complex Environmental Configuration	37.50
	Insufficient Basic Knowledge of Airflow	25
	Insufficient Knowledge of the External Systems	18.75
	Complex Workflow	6.25
(C7) Adoption of Airflow	Insufficient Basic Knowledge of Airflow	81.8
	Incorrect or Sub-optimal Operator Choice	9.09
	Complex Workflow	9.09

6. Discussion

In this section, we begin by presenting the connection between the root causes and challenges identified in RQ1 and RQ2. Then, we discuss the actionable implications of our findings for researchers and practitioners in the field. In the end, we compare our findings with related empirical studies on developers' challenges in other domains.

6.1. Implications

Table 6 provides a breakdown of the Top-4 most prevalent root causes associated with each of the seven high-level challenge types encountered by developers while developing and managing their workflows utilizing Airflow. Our main findings are:

- Three or more distinct root causes influence each of the seven challenge types, with the Top-4 root causes accounting for between 69% and 100% of the cases in each challenge type. This indicates a diverse range of underlying issues contributing to each challenge type.
- In six of the seven challenge categories, a single root cause is responsible for approximately half of the questions in that category. This contrasts with challenges related to quality assurance and workflow maintenance, where the root causes are more varied and spread across multiple factors.
- Insufficient knowledge about Airflow or external systems or Python is among the Top-4 root causes for all challenge types.
- Complex environmental configuration is the Top-1 root cause for four challenge types.

Based on Table 6, we make the following implications for different stakeholders.

6.1.1. Implications for researchers

Need research support on automated environment configuration.

Developers regularly encounter obstacles related to environmental configurations while working with Airflow. Table 6 illustrates that the intricacy of configuring environments is a principal factor that intensifies a variety of challenges, notably in "Environment Setup and Deployment" and "Security". This indicates a pressing need for future research aimed at aiding developers in the configuration of environments for workflows orchestrated by Airflow. Potential solutions could include the development of automated tools for configuring parameters or advanced debugging tools designed to streamline and simplify the configuration process. Prior studies (Xu et al., 2016) proposed approaches to tune configurations or detect configuration errors related to failure handling and fault tolerance of software systems. However, existing studies have not targeted the challenges in configuring workflows within data-intensive software systems, nor have they delved into the specifics of workflows orchestrated using Airflow.

Need research support on operator recommendation and configuration guidance.

Developers often face significant obstacles due to incorrect workflow configurations, identified as the primary root cause for the most common challenge, i.e., "Workflow Definition" as shown in Table 6. This issue, as explored in RQ1 and RQ2, often arises from the complexities involved in identifying appropriate operators and determining their optimal configurations. Addressing this challenge necessitates focused research on the development of sophisticated recommendation tools. Such tools would offer invaluable assistance to developers by steering them towards the selection and configuration of the most apt operators for their specific workflow needs.

6.1.2. Implications for Airflow application developers

Essential to gain a diverse skill set. Table 6 reveals that a lack of comprehensive knowledge in Airflow, external systems, and Python significantly contributes to various challenges. This highlights the critical need for developers to acquire a broad spectrum of skills within the Airflow ecosystem. Developers should enhance their expertise across

several domains, including Python scripting proficiency, a thorough understanding of the DAG structure, insights into distributed systems architecture, and a solid grasp of Airflow's diverse components. Adopting this knowledge enables developers to adeptly manage and leverage the full capabilities of Airflow, thereby streamlining workflow processes and overcoming the intricacies inherent in its operation.

Proactive monitoring and engagement in the Airflow community:

The ongoing development and improvement of Airflow are propelled by its dedicated community, which boasts 2,824 contributors.⁵⁷ Our findings from RQ2 highlight how breaking changes and bugs in specific releases can create obstacles for developers, especially when working with features that are not fully supported, such as dynamic workflows. To effectively overcome these challenges, we recommend that developers actively participate in the Airflow community. Engaging in forums, subscribing to mailing lists, and contributing to discussions are pivotal actions that not only ensure developers are up-to-date with the latest developments and practices but also promote a supportive and collaborative culture. This proactive involvement facilitates the sharing of valuable insights, strategies for addressing common issues, and guidance on maximizing the utility of Airflow's features. Through such dynamic engagement, developers can both contribute to and benefit from the collective wisdom and innovative progress of the Airflow ecosystem.

6.1.3. Implications for Airflow platform developers

Enable flexible operator design: RQ1 highlights that developers frequently face challenges in task configuration (C1.1), particularly with dynamic parameterization and the development of custom operators to expand Airflow's functionality. Additionally, the proliferation of similar operators, each with its own unique configuration options, has compounded these challenges. This situation underscores the necessity for a strategic overhaul of operator design within Airflow. A potential solution is to adopt a more modular and flexible architecture. Such an architecture would not only facilitate code reuse but also ensure adherence to a consistent API framework. By implementing a design philosophy that emphasizes modularity and extensibility, Airflow can greatly enhance the developer experience, making it easier to create and manage custom operators while promoting a more streamlined and efficient workflow development process.

Comprehensive support for shared data storage: Our findings in RQ1 show that incorrect workflow configuration (R1) is the primary root cause of challenges faced by developers, a significant portion of which can be traced back to Airflow's current limitations in offering sophisticated shared data storage solutions. The absence of robust shared data storage mechanisms, capable of seamless integration with external storage systems and facilitating efficient data exchange through mechanisms like XCom, compromises data consistency and reliability across tasks. To address these challenges, developers of Airflow could evolve its support for shared data storage. Enhancing its integration capabilities with external storage solutions and optimizing the efficiency of data exchange methods will be crucial steps forward. Such advancements will significantly elevate the utility and performance of Airflow in managing complex data workflows.

Need to enhance the official documentation of Airflow: While Airflow's official documentation is frequently cited in responses to Airflow-related inquiries (as shown in RQ3), developers encounter difficulties in locating specific information within it.⁵⁸ To address this, efforts are needed to improve the current official website of Airflow, especially to improve the description for the configuration of the built-in and providers' operators as well as environmental configurations. The goal should be to make it not only comprehensive but also intuitively organized, ensuring that researchers and developers can quickly find

the information they need to utilize Airflow's features effectively. This approach will significantly reduce the learning curve and improve the user experience.

A potential approach to improving Airflow documentation involves leveraging content from the Airflow repository. As evidenced in RQ3, components of the Airflow source code are frequently referenced, indicating their value and relevance. Large Language Models (LLMs) can be leveraged to extract and update documentation automatically based on repository changes. This approach could ensure that the documentation stays up-to-date with ongoing development, addressing an important concern among developers.

Moreover, enriching the documentation with best practices and real-world examples is vital. Such content can serve as a roadmap for developers, showing them how to leverage Airflow efficiently in their specific contexts. Practical demonstrations of Airflow in various use cases will provide valuable insights into workflow optimization, common problem resolutions, and innovative techniques for project enhancement. This strategy ensures that the documentation is not just informative but also actionable, helping users to exploit Airflow's capabilities in their operational environments fully.

6.2. Comparing to other Domains

Our taxonomy highlights a range of challenges exclusive to Workflows as Code, i.e., Task Definition and Configuration, Integration with External Systems, Data Sharing between Tasks, Workflow Design, Scheduling and Triggers, and Task Dependency.

There are intersections between our findings and the challenges identified in previous studies, particularly regarding security, testing, installation, performance optimization, maintenance, and logging within the context of Workflows as Code. Security concerns, as identified as one type of challenge in the configuration-as-code domain (Rahman et al., 2018), are crucial for safeguarding the integrity and confidentiality of workflow development. The testing challenge, discussed in the same work (Rahman et al., 2018), is essential for ensuring the reliability and functionality of workflow implementations. Additionally, installation challenges, highlighted in Deep Learning development (Zhang et al., 2019), deployment (Chen et al., 2020), and the configuration-as-code domain Rahman et al. (2018), are crucial for effectively setting up and configuring workflow platforms. Similar to the domains of Spark (Wang et al., 2022) and Big Data (Bagherzadeh and Khatchadourian, 2019), logging and performance optimization plays a vital role in capturing workflow-related logs and enhancing performance within the context of Workflows as Code.

It is important to note that among the studies mentioned, only Wang et al. (2022) analyzed the root causes behind these challenges, like our study is doing. Two of our identified root causes closely correspond with their findings: complex environmental configuration and a steep learning curve required for the target platform.

7. Threats to validity

External Validity. Threats to external validity are related to the generalization of our results. Our taxonomy of challenges and root causes associated with Airflow is based on a thorough analysis of 1000 randomly selected samples from a total of 9591 Stack Overflow (SO) posts tagged with Airflow-related terms. While we did not individually analyze each of the 9591 posts, we ensured statistical significance in our sample with a 95% confidence level and a 3% confidence interval. Similar to other studies that focus on understanding the challenges developers encounter within specific domains (Yang et al., 2016; Rosen and Shihab, 2016; Ahmed and Bagherzadeh, 2018; Alshangiti et al., 2019; Bagherzadeh and Khatchadourian, 2019; Alshangiti et al., 2019; Chen et al., 2020), our study exclusively relies on SO as the data source for investigating the challenges faced by developers, which may not be generalized to all kinds of challenges developers face while

⁵⁷ <https://github.com/apache/airflow>

⁵⁸ <https://stackoverflow.com/questions/74640349>

utilizing Airflow. In the future, we plan to validate our findings on other communication forums of Airflow-related practitioners.

Another threat to external validity arises from our choice of analysis target. As the initial effort to understand the challenges of implementing Workflows as Code, in this paper, we selected Apache Airflow from among the various platforms supporting such implementations. Our choice was primarily influenced by Airflow's widespread use in industry and its prominence on SO, where it has the highest number of question posts. In comparison, as of March 2024, the number of SO questions for other platforms is markedly lower: Luigi with 343 questions, Prefect with 192, Dagster with 171, Metaflow with 30, and Flyte with 19. Although these numbers are significantly smaller compared to Airflow, our analytical approach could be adapted for future research on these platforms, assuming more data becomes available.

Another threat to the external validity of our taxonomies is whether the taxonomy developed from posts collected before April 30, 2023, can be generalized to newer posts. To investigate this, we collected a recent dataset of Stack Overflow posts from May 1, 2023, to April 30, 2024, and filtered for posts with a non-negative score, resulting in 1,271 posts. We randomly selected 100 posts from these data and labeled them according to our existing taxonomies of challenges and root causes. Our results find that the tasks fall into the following high-level challenges: "Workflow Definition" (36 cases), "Workflow Execution" (24 cases), "Environment Setup and Deployment" (19 cases), "Quality Assurance and Maintenance" (15 cases), "Security" (4 cases), with 2 cases irrelevant. The sub-categories include Task Definition and Configuration (20), Task Dependency (14), Logging and Monitoring (14), Scheduling and Triggers (10), Installation (8), Data Sharing between Tasks (7), Deployment (6), Workflow Design (6), Component Setup and Execution (4), Integration with External Systems (3), Modules Management (1), and Airflow Version Update (1). Our findings indicate that these recent posts fit well within our challenge taxonomy, although the number of categories in the sample (5 high-level and 12 sub-categories) is slightly less than our full taxonomy (7 high-level and 14 sub-categories). Additionally, we identified the following root causes in the new samples that include Incorrect Workflow Configuration (19 cases), Insufficient Basic Knowledge of Airflow (12 cases), Complex Environmental Configuration (11 cases), Lack of Knowledge in Programming and DevOps (10 cases), Insufficient Knowledge of External Systems (10 cases), Limitations in Airflow (9 cases), Complex Workflows (7 cases), Missing or Incorrect Dependency (6 cases), and Incorrect or Sub-optimal Operator Choice (4 cases), and Unclear (12 cases). Our taxonomy includes a total of 10 root causes, and the new samples are covered by 9 of these causes. These results suggest that our taxonomy remains applicable to recent data, thereby supporting its generalizability and stability over time.

Internal Validity. Threats to internal validity are related to experiment errors or biases. In addressing RQ1 and RQ2, we carefully annotated 1,000 SO question posts to construct two comprehensive taxonomies: one for categorizing types of challenges related to Airflow and the other for identifying their underlying root causes. To mitigate potential bias and errors introduced by the labeling process, we employed a robust two-step open coding process. Initially, two authors collaboratively labeled a representative pilot set (30%) of the data. Following this, they independently analyzed and labeled the remaining 70% posts. This approach allowed for the cross-validation of findings. Whenever discrepancies emerged in the labeling process, two annotators discussed and resolved them jointly. We computed the Cohen's Kappa and found the agreement scores to be sufficiently high (0.81 and 0.79). When measuring the popularity and difficulty of the Airflow-related questions studied in the preliminary study, we opted to follow the same set of measurement metrics used in previous studies (Yang et al., 2016; Rosen and Shihab, 2016; Ahmed and Bagherzadeh, 2018; Alshangiti et al., 2019; Bagherzadeh and Khatchadourian, 2019). While these metrics provide a structured approach to our analysis, it is important to acknowledge that they might not comprehensively capture all aspects of question popularity and difficulty.

8. Conclusion

Workflows as Code simplifies the complexities of orchestrating workflows in data-intensive software systems. In this study, we gathered 9591 Q&A posts related to the leading Workflows as Code platform, Airflow from Stack Overflow and carried out an empirical analysis to explore the challenges and root causes encountered by developers using Airflow, as well as to examine the external resources shared through URLs within these questions.

Manually analyzing 1,000 sampled posts, we identified 7 high-level categories and 14 sub-categories for Airflow-related challenges, with the most significant challenge lying in "Workflow Definition", accounting for 36.6% of the identified challenges. Through our analysis, we identified ten root causes underlying these challenges, ranging from incorrect workflow configuration to a lack of basic knowledge of Airflow. We also found that external resources are frequently shared in answers to Airflow-related questions, which underscores the importance of diverse knowledge required to resolve Airflow-related challenges that developers face.

Our findings provide insights into the practical difficulties encountered when implementing Workflows as Code, underlying the necessity for improved support and a more profound comprehension of this paradigm in the software engineering field.

CRedit authorship contribution statement

Jerin Yasmin: Conceptualization, Data curation, Investigation, Methodology, Project administration, Validation, Writing – original draft, Writing – review & editing. **Jiale Amber Wang:** Data curation, Investigation, Validation, Writing – original draft. **Yuan Tian:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization. **Bram Adams:** Conceptualization, Supervision, Validation, Writing – original draft.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yuan Tian reports financial support was provided by Natural Sciences and Engineering Research Council of Canada. Yuan Tian reports a relationship with Natural Sciences and Engineering Research Council of Canada that includes: funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC) [funding reference number: RGPIN-2019-05071].

Data availability

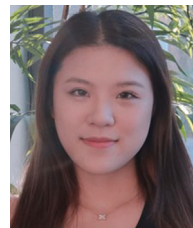
The results, source code, and data related to this study are available at https://github.com/RISElabQueens/dataset_developers_challenges_WaC.

References

- Ahmed, S., Bagherzadeh, M., 2018. What do concurrency developers ask about? a large-scale study using stack overflow. In: *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. pp. 1–10.
- Airflow Documentation, 2024a. Python operator. URL: <https://airflow.apache.org/docs/apache-airflow/stable/howto/operator/python.html>.
- Airflow Documentation, 2024b. TaskFlow. URL: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/taskflow.html>.
- Allamanis, M., Sutton, C., 2013. Why, when, and what: analyzing stack overflow questions by topic, type, and code. In: *2013 10th Working Conference on Mining Software Repositories*. MSR, IEEE, pp. 53–56.
- Alshangiti, M., Sapkota, H., Murukannaiah, P.K., Liu, X., Yu, Q., 2019. Why is developing machine learning applications challenging? a study on stack overflow posts. In: *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (Esem)*. IEEE, pp. 1–11.
- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., Zimmermann, T., 2019. Software engineering for machine learning: A case study. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, pp. 291–300.
- Bagherzadeh, M., Khatchadourian, R., 2019. Going big: a large-scale study on what big data developers ask. In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 432–442.
- Baltes, S., Dumani, L., Treude, C., Diehl, S., 2018. Sotorrent: reconstructing and analyzing the evolution of stack overflow posts. In: *Proceedings of the 15th International Conference on Mining Software Repositories*. pp. 319–330.
- Berg, D., Chirravuri, R., Cledat, R., Goyal, S., Hamad, F., Tuulos, V., 2019. Open-sourcing metaflow, a human-centric framework for data science. *Netflix Tech. Blog*. 201.
- Beyer, S., Macho, C., Di Penta, M., Pinzger, M., 2020. What kind of questions do developers ask on Stack Overflow? A comparison of automated approaches to classify posts into question categories. *Empir. Softw. Eng.* 25, 2258–2301.
- C. W. Danilo Sato, A.W., 2019. Continuous delivery for machine learning. URL: <https://martinfowler.com/articles/cd4ml.html>.
- Chen, Z., Cao, Y., Liu, Y., Wang, H., Xie, T., Liu, X., 2020. A comprehensive study on challenges in deploying deep learning based software. In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 750–762.
- Cummaudo, A., Vasa, R., Barnett, S., Grundy, J., Abdelrazek, M., 2020. Interpreting cloud computer vision pain-points: a mining study of Stack Overflow. In: *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. IEEE.
- John Thomas, E.T., 2022. Airflow survey 2022 | apache airflow. URL: <https://airflow.apache.org/blog/airflow-survey-2022>.
- Kersting, K., Kim, M., Van den Broeck, G., Zimmermann, T., 2020. SE4ML-software engineering for AI-ML-based systems (dagstuhl seminar 20091). In: *Dagstuhl Reports*, vol. 10. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- Landis, J.R., Koch, G.G., 1977. The measurement of observer agreement for categorical data. *Biometrics* 159–174.
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I.D., Gebru, T., 2019. Model cards for model reporting. In: *Proceedings of the Conference on Fairness, Accountability, and Transparency*. pp. 220–229.
- Munappy, A.R., Bosch, J., Olsson, H.H., 2020. Data pipeline management in practice: Challenges and opportunities. In: *International Conference on Product-Focused Software Process Improvement*. Springer, pp. 168–184.
- Munappy, A., Bosch, J., Olsson, H.H., Arpteg, A., Brinne, B., 2019. Data management challenges for deep learning. In: *2019 45th Euromicro Conference on Software Engineering and Advanced Applications*. SEAA, IEEE, pp. 140–147.
- Novella, J.A., Emami Khoonsari, P., Herman, S., Whitenack, D., Capuccini, M., Burman, J., Kultima, K., Spjuth, O., 2019. Container-based bioinformatics with Pachyderm. *Bioinformatics* 35 (5), 839–846.
- Polyzotis, N., Roy, S., Whang, S.E., Zinkevich, M., 2018. Data lifecycle challenges in production machine learning: a survey. *ACM SIGMOD Record* 47 (2), 17–28.
- Ponzanelli, L., Mucci, A., Bacchelli, A., Lanza, M., 2014. Understanding and classifying the quality of technical forum questions. In: *2014 14th International Conference on Quality Software*. IEEE, pp. 343–352.
- Rahman, A., Partho, A., Morrison, P., Williams, L., 2018. What questions do programmers ask about configuration as code? In: *Proceedings of the 4th International Workshop on Rapid Continuous Software Engineering*. pp. 16–22.
- Rosen, C., Shihab, E., 2016. What are mobile developers asking about? a large scale study using stack overflow. *Empir. Softw. Eng.* 21, 1192–1223.
- Sally, 2019. The apache software foundation blog. URL: <https://news.apache.org/foundation/entry/the-apache-software-foundation-announces44>.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., 2014. Machine learning: The high interest credit card of technical debt. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D., 2015. Hidden technical debt in machine learning systems. *Adv. Neural Inf. Process. Syst.* 28.
- Seaman, C.B., 1999. Qualitative methods in empirical studies of software engineering. *IEEE Trans. Softw. Eng.* 25 (4), 557–572.
- Star-History, 2024. GitHub star history. URL: <https://star-history.com/#spotify/luigi&argoproj/argo&kubeflow/kubeflow&mlflow/mlflow&apache/airflow&PrefectHQ/prefect&dagster-io/dagster&Date>.
- Tagliabue, J., Tuulos, V., Greco, C., Dave, V., 2021. DAG card is the new model card. *arXiv preprint arXiv:2110.13601*.
- Viera, A.J., Garrett, J.M., et al., 2005. Understanding interobserver agreement: the kappa statistic. *Fam. Med.* 37 (5), 360–363.
- Wang, Z., Chen, T.-H.P., Zhang, H., Wang, S., 2022. An empirical study on the challenges that developers encounter when developing Apache Spark applications. *J. Syst. Softw.* 194, 111488.
- Xu, T., Jin, X., Huang, P., Zhou, Y., Lu, S., Jin, L., Pasupathy, S., 2016. Early detection of configuration errors to reduce failure damage. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. pp. 619–634.
- Yackel, R., 2021. IBM blog. URL: <https://www.ibm.com/blog/6-issues-with-airflow/>.
- Yang, X.-L., Lo, D., Xia, X., Wan, Z.-Y., Sun, J.-L., 2016. What security questions do developers ask? a large-scale study of stack overflow posts. *J. Comput. Sci. Tech.* 31, 910–924.
- Zhang, T., Gao, C., Ma, L., Lyu, M., Kim, M., 2019. An empirical study of common challenges in developing deep learning applications. In: *2019 IEEE 30th International Symposium on Software Reliability Engineering*. ISSRE, IEEE, pp. 104–115.



Jerin Yasmin is a Ph.D. student at Queen's University, Canada. Her research focuses on software engineering for AI (SE4AI), where she works on understanding and improving data and ML workflows in software systems. Her research interests include AI4SE, Mining Software Repositories, and Software Maintenance and Evolution. Her work has been published in top-tier software engineering venues such as ICSME, ICPC and MSR. Before starting her graduate studies at Queen's, Jerin gained industry experience in back-end architecture, web services development, and security enhancements.



Jiale Amber Wang is a Master's student at the University of Waterloo. She earned her bachelor's degree in Computing and Mathematics from Queen's University, Canada. Jiale's research focuses on applying machine learning techniques in software engineering to enhance developer efficiency, with particular interest in software testing automation, natural language processing in code analysis, and leveraging data-driven approaches to optimize software development processes.



Yuan Tian obtained her Ph.D. from Singapore Management University, Singapore. Currently, she is an Assistant Professor at Queen's University. She leads the RISE research lab, focusing on reliable and intelligent support for software development. Her work has been published in top-tier software engineering venues such as ICSE, ASE, TSE, and TOSEM. Her work on automated bug report prioritization won the Most Influential Paper award at SANER 2022.



Bram Adams obtained his Ph.D. from Ghent University, Belgium. Currently, he is a full professor at Queen's University. His research interests include software release engineering, software engineering for AI (and vice versa), and mining software repositories. His work earned the 2021 MSR Foundational Contribution Award and has been published at premier software engineering venues such as ICSE, FSE, MSR, ICSME, EMSE and TSE. He has co-organized numerous prominent events and was PC co-chair for multiple conferences. He is a Senior IEEE Member.